Bellman Ford

```c
#include <stdio.h>
#include <stdlib.h>

#define INFINITY 99999
typedef struct Edge {
  int u;
  int v;
  int w;
}e;

typedef struct Graph {
  int V;
  int E;
  struct Edge *edge;
}gr;

void display(int arr[], int size) {
  for (int i = 1; i <= size; i++)
  {
    printf("%d ", arr[i]);
  }
}

void bellmanford(gr g, int source)
{
  int i, j, u, v, w;
  int tV = g.V;
  int tE = g.E;
  int d[tV];
  for (i = 1; i <= tV; i++)
  {
    d[i] = INFINITY;
  }
  d[source] = 0;
  for (i = 2; i <=tV ; i++)
  {
    for (j = 0; j < tE; j++)
    {
      u = g.edge[j].u;
      v = g.edge[j].v;
      w = g.edge[j].w;

      if (d[u] != INFINITY && d[v] > d[u] + w)
      {
        d[v] = d[u] + w;
      }
```

```
    }
  }

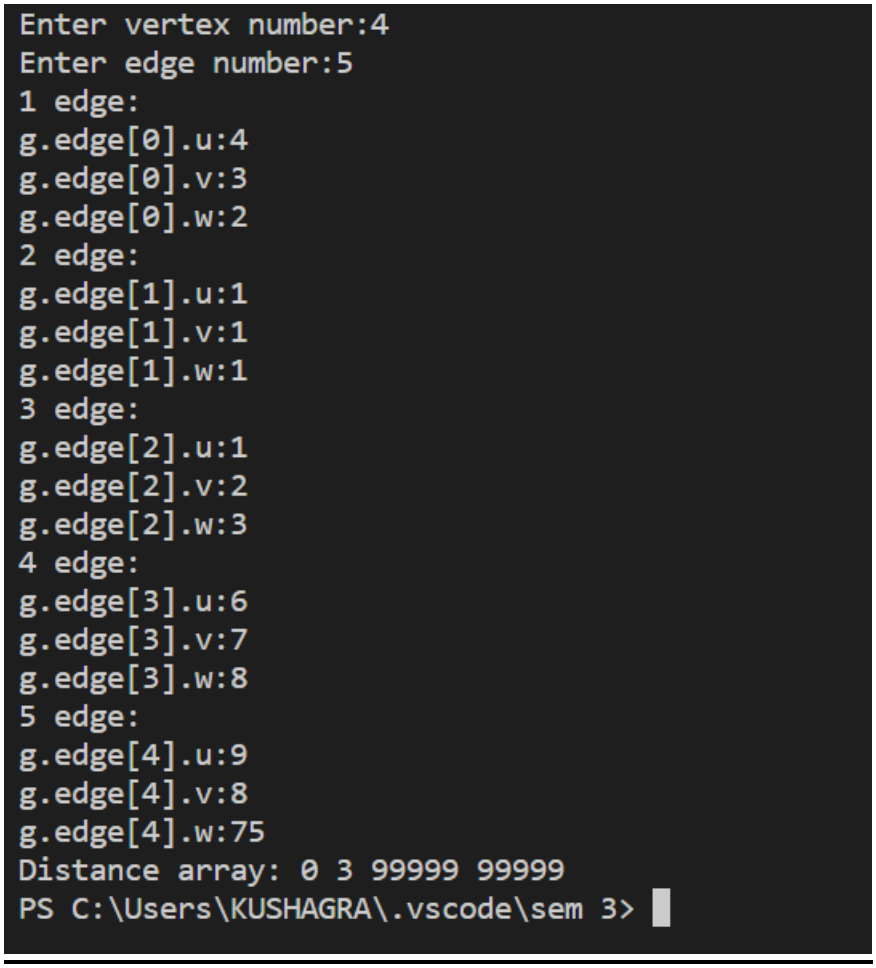
  for (i = 0; i < tE; i++)
  {
    u = g.edge[i].u;
    v = g.edge[i].v;
    w = g.edge[i].w;
    if (d[u] != INFINITY && d[v] > d[u] + w)
    {
      printf("Negative weight cycle detected!\n");
      return;
    }
  }
  printf("Distance array: ");
  display(d, tV);
}

int main()
{
  gr g;
  printf("Enter vertex number:");
  scanf("%d",&g.V);
  printf("Enter edge number:");
  scanf("%d",&g.E);
  g.edge =(e *)malloc(g.E * sizeof(e));

  for (int i = 0; i < g.E; i++)
  {
      printf("%d edge:\n",i+1);
      printf("g.edge[%d].u:",i);
      scanf("%d",&g.edge[i].u);
      printf("g.edge[%d].v:",i);
      scanf("%d",&g.edge[i].v);
      printf("g.edge[%d].w:",i);
      scanf("%d",&g.edge[i].w);
  }
  bellmanford(g, 1);

  return 0;
}
```

Output:


Enter vertex number:4
Enter edge number:5
1 edge:
g.edge[0].u:4
g.edge[0].v:3
g.edge[0].w:2
2 edge:
g.edge[1].u:1
g.edge[1].v:1
g.edge[1].w:1
3 edge:
g.edge[2].u:1
g.edge[2].v:2
g.edge[2].w:3
4 edge:
g.edge[3].u:6
g.edge[3].v:7
g.edge[3].w:8
5 edge:
g.edge[4].u:9
g.edge[4].v:8
g.edge[4].w:75
Distance array: 0 3 99999 99999
PS C:\Users\KUSHAGRA\.vscode\sem 3>